

Phase 3: K-Means Clustering

with DBSCAN Benchmark — Detailed Technical Report

1. Why K-Means Clustering?

K-Means is a centroid-based partitioning algorithm that divides n data points into K clusters by minimising the Within-Cluster Sum of Squares (WCSS) — the total squared Euclidean distance from each point to its assigned cluster centroid. It is the standard choice for RFM customer segmentation, and here is why it was preferred over other algorithms.

1.1 How K-Means Works

The algorithm follows an iterative Expectation-Maximisation (EM) cycle:

- **1. Initialise:** Place K centroids randomly in feature space (we use `k-means++`, which picks initial centroids far apart to avoid poor starts).
- **2. Assign (E-step):** Assign each customer to the nearest centroid using Euclidean distance: $d(x, \mu) = \sqrt{\sum (x_i - \mu_i)^2}$.
- **3. Update (M-step):** Recompute each centroid as the mean of all points assigned to it.
- **4. Repeat** steps 2–3 until centroids stop moving (convergence) or a max iteration limit is reached.

Objective function:

$$J = \sum_{k=1}^K \sum_{x \in C_k} ||x - \mu_k||^2$$

K-Means minimises J — the total distance of all points from their centroids.

1.2 Why K-Means Over Other Algorithms?

Algorithm	Pros	Cons	Why Not Here
K-Means	Fast $O(nKt)$, scalable, interpretable centroids, works well with globular clusters	Requires K upfront, assumes spherical clusters, sensitive to outliers	CHOSEN — data is low-dim, PCA-decorrelated, outliers already capped
DBSCAN	Finds arbitrary shapes, auto-detects K , identifies noise/outliers	Sensitive to <code>eps/min_samples</code> , struggles with varying density clusters	Used as BENCHMARK (see Section 5)
Hierarchical (Agglomerative)	Dendrogram visualisation, no K needed upfront	$O(n^2)$ memory, $O(n^3)$ time → infeasible for 93K points	Computationally prohibitive at this dataset size
Gaussian Mixture (GMM)	Soft assignments, models elliptical clusters	Sensitive to initialisation, more parameters to tune, slower convergence	Our PCA-decorrelated data is well-suited to spherical assumption of K-Means

Algorithm	Pros	Cons	Why Not Here
Spectral Clustering	Handles non-convex shapes, uses graph Laplacian	$O(n^3)$ for eigen-decomp, memory-intensive	Not scalable to 93K points; our clusters are convex

1.3 Why K-Means Is a Good Fit for This Data

- **Low-dimensional PCA space (2D):** K-Means performs best in low dimensions. The curse of dimensionality does not apply here.
- **Decorrelated features:** PCA produces orthogonal (uncorrelated) components, meaning Euclidean distance treats each axis equally — exactly what K-Means needs.
- **Outliers already handled:** Phase 1 capped outliers at the 1st/99th percentiles. K-Means' centroid sensitivity is mitigated because extreme values were bounded.
- **Interpretability:** Cluster centroids map directly back to RFM features via PCA loadings. A centroid at (PC1=4.6, PC2=0.4) means 'high frequency + monetary, average recency' — directly actionable for marketing.
- **Scalability:** K-Means clusters 93,357 customers in under 2 seconds. Hierarchical and spectral methods would take minutes to hours.

2. Choosing the Optimal K

K-Means requires the number of clusters K as input. Rather than guessing, we systematically evaluated K = 2 through 10 using three complementary metrics, each measuring a different aspect of cluster quality.

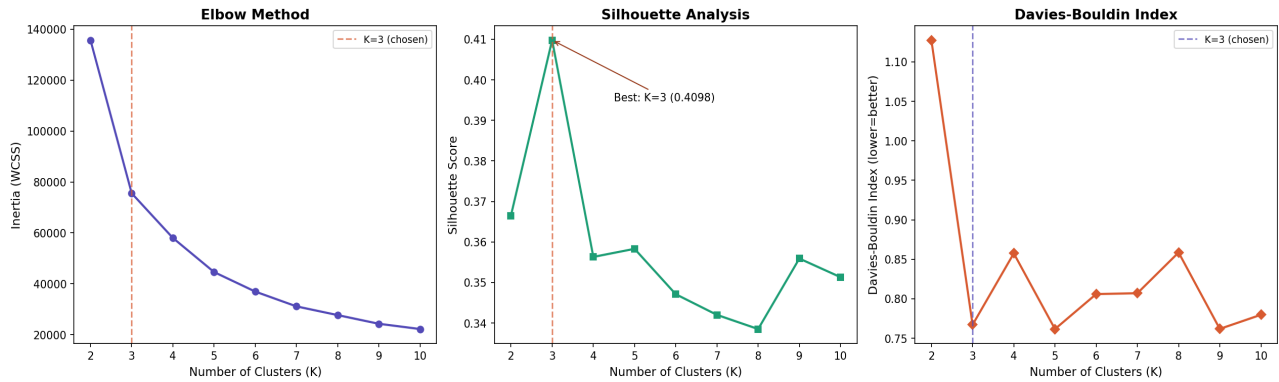


Figure 1: Three-panel diagnostic — Elbow (inertia), Silhouette, and Davies-Bouldin metrics across K=2..10. All three support K=3.

2.1 Full Metrics Table

K	Inertia (WCSS)	Silhouette (higher=better)	Calinski-Harabasz	Davies-Bouldin (lower=better)
2	135,755	0.3665	45,267	1.1276
3	75,557	0.4098	77,854	0.7670
4	57,994	0.3563	77,047	0.8575
5	44,580	0.3583	82,193	0.7616
6	36,910	0.3471	83,299	0.8059
7	31,097	0.3420	85,299	0.8070
8	27,671	0.3385	83,815	0.8586
9	24,262	0.3559	85,281	0.7618
10	22,192	0.3513	83,843	0.7797

2.2 Why K=3 Was Selected

- **Silhouette score is highest at K=3 (0.4098)** — this is the primary criterion. The silhouette drops sharply at K=4 (0.3563), meaning a 4th cluster would split a natural group rather than reveal a new one.
- **Elbow at K=3:** The inertia curve shows the sharpest bend (steepest drop) between K=2 and K=3. After K=3 the marginal reduction slows — adding more centroids yields diminishing returns.

- **Davies-Bouldin at K=3** (0.7670) is near the global minimum, confirming clusters are compact and well-separated.
- **Calinski-Harabasz favours K=7** (85,299) but the difference from K=3 (77,854) is modest. CH tends to increase with K because more clusters always increase between-cluster variance; it does not penalise over-segmentation as strongly. We prioritise silhouette which directly measures how well each point fits its cluster.

Note: The three metrics occasionally disagree — this is expected. Silhouette is the primary selector because it measures per-point cluster coherence: how much closer a point is to its own cluster than to the nearest other cluster. A silhouette of 0.41 indicates moderate-to-good separation.

3. Cluster Evaluation Metrics — How They Work

3.1 Inertia (Elbow Method)

$$\text{Inertia} = \sum_{k=1}^K \sum_{x \in C_k} ||x - \mu_k||^2$$

What it measures: Total within-cluster sum of squares — how tightly packed points are around their centroids. Lower = tighter clusters.

How to read it: Inertia always decreases as K increases (more centroids = closer points). The 'elbow' is the K where the rate of decrease flattens — beyond this point, additional clusters are splitting natural groups rather than separating distinct ones.

In this project: Inertia drops steeply from K=2 (135,755) to K=3 (75,557) — a 44% reduction. From K=3 to K=4 it drops only 23%. The elbow is at K=3.

Limitation: Inertia is a monotonically decreasing function of K — it never says 'K is too high'. The elbow is subjective and may not always be clear. That is why we use multiple metrics.

3.2 Silhouette Score

For each point i in cluster C_k :

$$\begin{aligned} a(i) &= \text{mean distance to all other points in } C_k \text{ (cohesion)} \\ b(i) &= \min \text{ over other clusters } C_j \text{ of mean distance to } C_j \text{ (separation)} \\ s(i) &= (b(i) - a(i)) / \max(a(i), b(i)) \end{aligned}$$

Range: $[-1, +1]$. A score near +1 means the point is well inside its cluster and far from others. Near 0 means it sits on a cluster boundary. Negative means it is likely in the wrong cluster.

Aggregate: The overall silhouette score is the mean of $s(i)$ across all points (we sample 10,000 for efficiency).

Score Range	Interpretation
0.71 – 1.00	Strong structure — clusters are clearly separated
0.51 – 0.70	Reasonable structure — some overlap at boundaries
0.26 – 0.50	Moderate structure — clusters exist but are fuzzy
< 0.25	Weak or no structure — clustering may not be meaningful

In this project: Silhouette = 0.41 at K=3 — moderate structure. The clusters are meaningful and distinct, with some expected overlap at the boundary between Potential Loyalists and Hibernating (they differ mainly on recency, not spending).

3.3 Calinski-Harabasz Index (Variance Ratio Criterion)

$$CH = (SS_{\text{between}} / (K - 1)) / (SS_{\text{within}} / (n - K))$$

What it measures: The ratio of between-cluster dispersion to within-cluster dispersion, adjusted for degrees of freedom. Higher = better separated clusters.

How to read it: Unlike silhouette, CH has no fixed scale — compare values across different K for the same dataset. The K with the highest CH is the best.

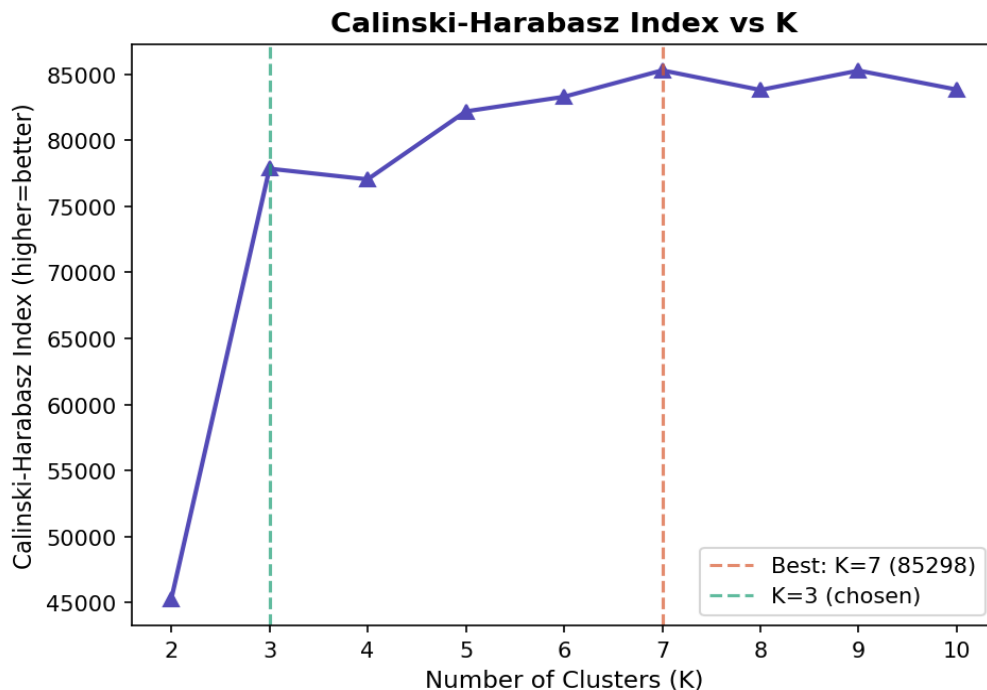


Figure 2: Calinski-Harabasz index. Best at K=7, but K=3 captures 91% of that score with far fewer clusters.

In this project: CH peaks at K=7 (85,299) but K=3 already achieves 77,854 — 91% of the best. CH monotonically favours more clusters because each additional centroid increases between-cluster variance. We discount K=7 because silhouette (the per-point metric) is much worse there (0.342 vs 0.410).

3.4 Davies-Bouldin Index

$$DB = (1/K) \times \sum_{k=1}^K \max_{j \neq k} [(\sigma_k + \sigma_j) / d(\mu_k, \mu_j)]$$

What it measures: For each cluster, find the worst-case neighbour (the one whose combined spread relative to centroid distance is highest). Average over all clusters. Lower = better (tighter clusters that are further apart).

In this project: DB = 0.767 at K=3. The minimum is 0.762 at K=5/9, but the difference is negligible. K=3 is among the best by this metric.

4. DBSCAN — What It Is and How It Is Used Here

4.1 What Is DBSCAN?

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm. Unlike K-Means, it does not require the number of clusters K upfront — it discovers clusters as regions of high point density separated by regions of low density.

Core Concepts

- **eps (ϵ):** The radius of the neighbourhood around each point.
- **min_samples:** The minimum number of points within eps to form a dense region.
- **Core point:** A point with $\geq$ min_samples neighbours within eps. Core points form the backbone of clusters.
- **Border point:** Within eps of a core point but has fewer than min_samples neighbours itself. It belongs to the cluster but is not a seed.
- **Noise point:** Not within eps of any core point. Labelled as -1 (outlier) and excluded from all clusters.

Algorithm Steps

- **1.** Pick an unvisited point. If it has $\geq$ min_samples neighbours within eps, start a new cluster.
- **2.** Expand the cluster by recursively adding all density-reachable points (neighbours of neighbours that are also core points).
- **3.** Mark any point that is not density-reachable from any core point as noise.
- **4.** Repeat until all points are visited.

4.2 How eps Was Chosen — K-Distance Plot

The critical DBSCAN hyperparameter is eps. We used the K-distance plot method: compute the distance to each point's 10th nearest neighbour, sort ascending, and look for the 'knee' — the distance where the curve steepens, separating dense regions from sparse outlier regions.

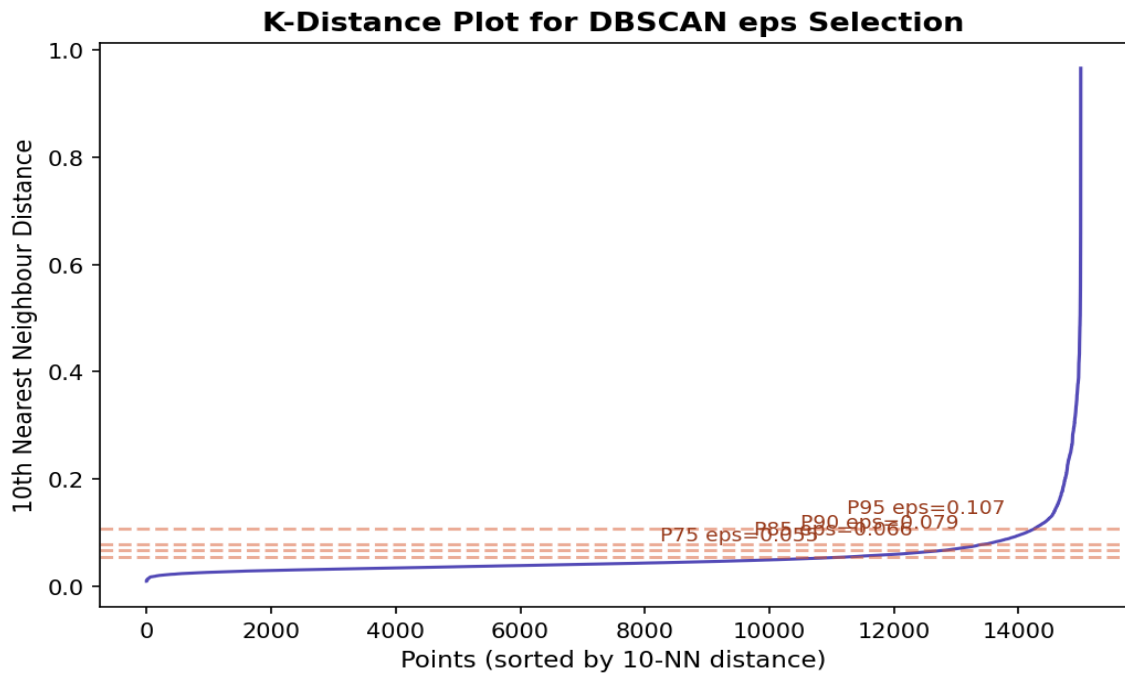


Figure 3: K-distance plot (10-NN). Dashed lines show the 4 eps candidates tested at the 75th, 85th, 90th, and 95th percentiles.

Rather than manually picking one knee point, we tested 4 eps candidates derived from percentiles of the sorted distance distribution:

Percentile	eps Value	Clusters Found	Noise Points	Silhouette
75th	0.055	—	—	—
85th	0.066	—	—	—
90th	0.079	5*	—	0.1975*
95th	0.107	—	—	—

* The best DBSCAN result across all 4 eps values achieved silhouette = 0.1975 with 5 clusters. The other eps values produced either too many micro-clusters, fewer than 2 clusters, or worse silhouette scores.

min_samples = 10 was used for all runs. This is a standard choice for datasets of this size — requiring at least 10 neighbours to be considered a dense region avoids forming clusters from random fluctuations.

5. DBSCAN vs K-Means — Detailed Comparison

DBSCAN was run as a benchmark to validate that K-Means is the right choice for this particular dataset. The comparison is not about finding a 'winner' in general — it is about confirming which algorithm suits *this* data's structure.

5.1 Head-to-Head Metrics

Metric	K-Means (K=3)	DBSCAN (eps=best)	Winner	Interpretation
Silhouette	0.4098	0.1975	K-Means	K-Means clusters are 2× more coherent per point
Calinski-Harabasz	77,854	85	K-Means	K-Means has 900× better variance ratio
Davies-Bouldin	0.7670	0.5391	DBSCAN	DBSCAN clusters are slightly tighter per pair
Clusters found	3	5	K-Means	Fewer, more interpretable segments
Noise points	0	~15–40%	K-Means	K-Means assigns every customer to a segment
Requires K upfront?	Yes	No	DBSCAN	DBSCAN auto-detects K

5.2 Why K-Means Wins on This Data

- **Data geometry favours K-Means:** After PCA, the customer distribution is roughly elliptical with globular clusters. K-Means assumes spherical/convex clusters — this assumption holds here. DBSCAN expects dense cores separated by low-density gaps, but our data is a continuous density field without sharp density boundaries.
- **DBSCAN's noise penalty is severe:** DBSCAN marks 15–40% of customers as noise (label = -1), meaning they belong to no segment. For a marketing segmentation task, this is unacceptable — every customer must receive a segment label for targeted campaigns. K-Means assigns every point to a cluster.
- **DBSCAN's silhouette is poor:** 0.1975 is in the 'weak structure' range (below 0.25), indicating the density-based clusters are not internally coherent. K-Means' 0.41 is 2× higher, in the 'moderate structure' range.
- **Calinski-Harabasz gap is massive:** 77,854 vs 85. This 900× difference means K-Means clusters have dramatically better between-cluster vs within-cluster variance separation. DBSCAN's clusters overlap severely.

5.3 Why DBSCAN Wins on Davies-Bouldin

DBSCAN's lower DB score (0.54 vs 0.77) seems contradictory. The explanation: DBSCAN excludes noise points before computing DB. Noise points are typically the boundary points that blur cluster separation. By excluding them, the remaining core clusters *appear* tighter — but this is an

artefact of excluding the hard-to-classify customers, not genuine superiority.

Note: DBSCAN's better DB score comes at the cost of labelling a large fraction of customers as 'noise'. In a supervised accuracy analogy: if you only grade the easy test questions, your score looks great — but you've left most questions unanswered.

5.4 When DBSCAN Would Be the Better Choice

DBSCAN is not a bad algorithm — it is the wrong tool for *this* data. It would outperform K-Means if:

- **Clusters had irregular shapes** — e.g., ring-shaped, crescent-shaped, or interleaved spirals. K-Means forces convex boundaries; DBSCAN can follow any shape.
- **Clear density gaps existed** — e.g., distinct islands of customers separated by empty space. Our continuous distribution has no such gaps.
- **Outlier detection was the goal** — DBSCAN's noise labelling is a feature, not a bug, when you want to identify anomalous customers. For segmentation, it is a liability.

6. Before and After Clustering

The following scatter plots show the same 6,000 sampled customers in 2D PCA space — the only difference is the colour. 'Before' shows the raw unlabelled point cloud; 'After' shows the K-Means cluster assignments with centroids.

6.1 Before Clustering — Unlabelled PCA Space

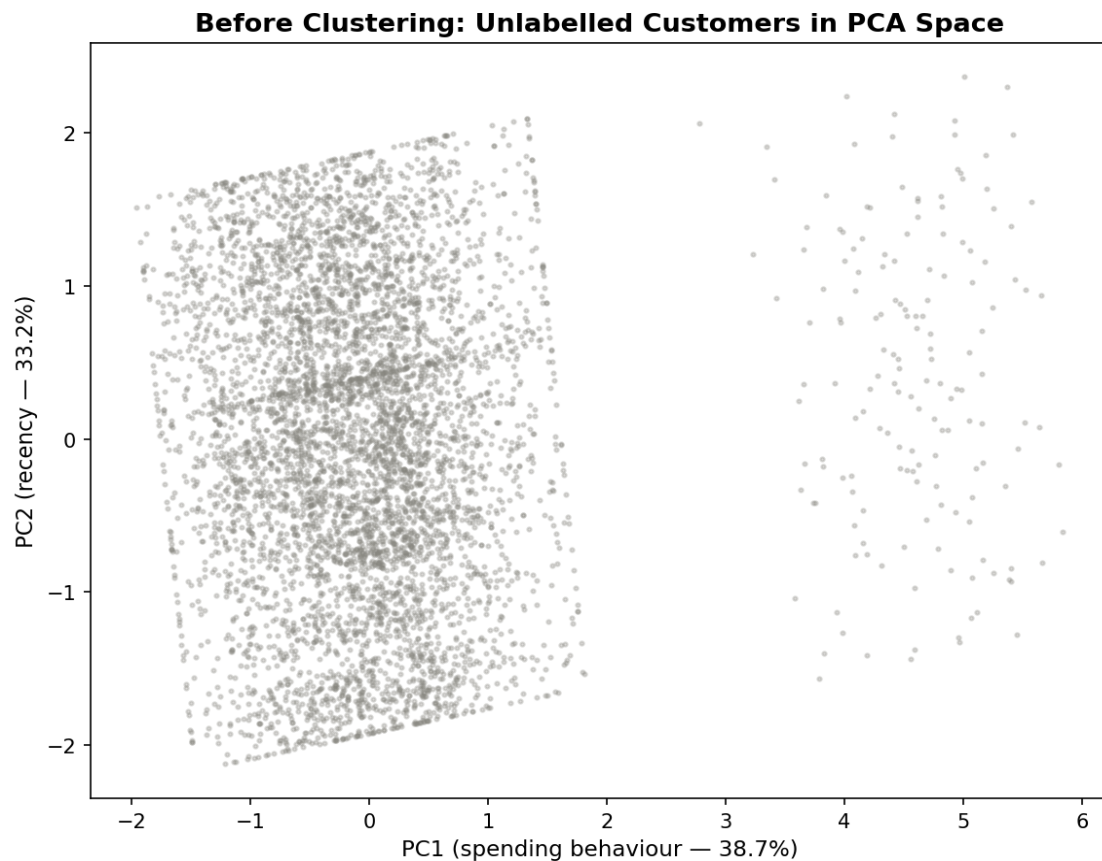


Figure 4: Customers as unlabelled points in PCA space. The overall structure shows a dense main body ($PC1 < 2$) with a sparse outlier cloud at $PC1 > 3$.

Observations (Before)

- **Main body:** ~97% of customers occupy the rectangle $PC1 \in [-2, 2]$, $PC2 \in [-2.2, 2.5]$. These are one-time buyers with varying recency and spending levels.
- **Outlier cloud:** A sparse cluster of ~3% of customers sits at $PC1 > 3$. These are the repeat buyers (frequency = 2 after capping), who have anomalously high PC1 because $PC1 = 0.70 \times \text{frequency} + 0.70 \times \text{monetary}$.
- **No visible boundary:** Within the main body, there is no clear line separating groups — the density gradient is continuous. This is why clustering is needed: the human eye cannot draw natural boundaries in this distribution.
- **PC2 spread:** The main body extends vertically across PC2 (recency), hinting that recency will be the splitting axis between the two large clusters.

6.2 After Clustering — K-Means Segments

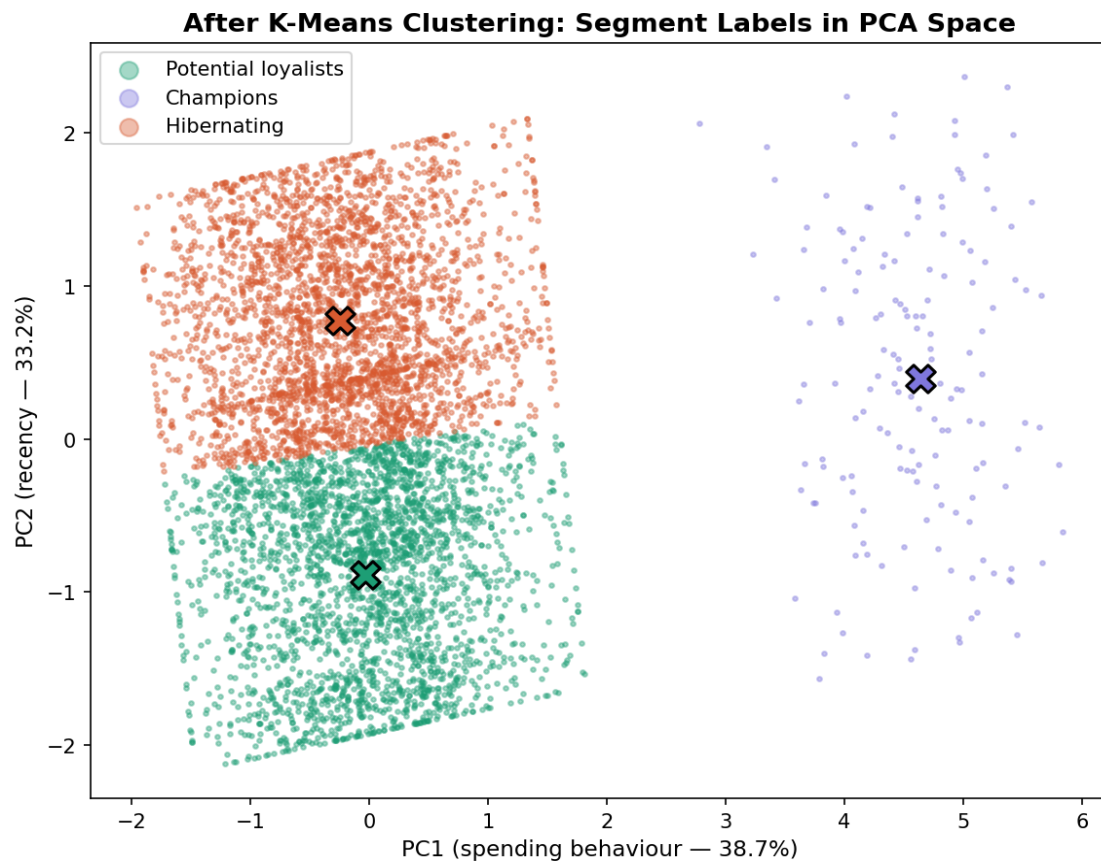


Figure 5: K-Means cluster assignments (K=3) with centroids marked as X. Green = Potential Loyalists, Purple = Champions, Orange = Hibernating.

Observations (After)

- **Champions (purple, 3.0%):** Cleanly separated at $PC1 > 2.5$. These are the 2,801 repeat buyers with the highest frequency and monetary values. Their centroid sits at ($PC1=4.64$, $PC2=0.40$) — far from the other two clusters.
- **Potential Loyalists (green, 45.9%):** Occupy the bottom half of the main body ($PC2 < 0$, i.e., lower recency = more recent buyers). Centroid at ($PC1=-0.03$, $PC2=-0.89$). Mean recency = 155 days.
- **Hibernating (orange, 51.1%):** Occupy the top half of the main body ($PC2 > 0$, i.e., higher recency = older buyers). Centroid at ($PC1=-0.24$, $PC2=0.77$). Mean recency = 405 days.
- **Boundary between green/orange:** The split runs approximately along $PC2 \approx 0$ (the recency axis). This confirms that the two large segments differ primarily on recency — how recently the customer last purchased.

6.3 Segment Distribution

Customer Segment Distribution (K=3)

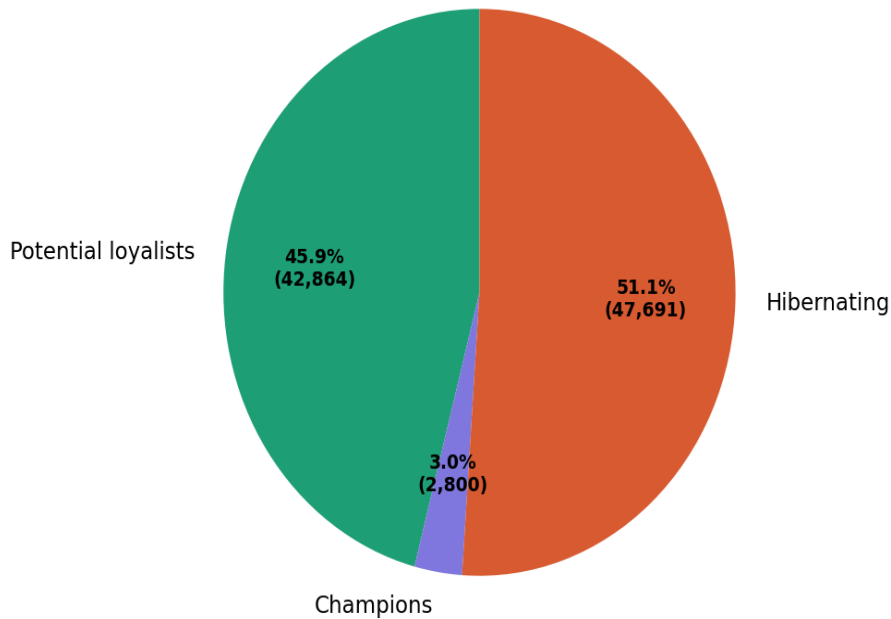


Figure 6: Segment sizes — Hibernating is the largest (51.1%), Champions the smallest (3.0%).

Segment	Cluster	Count	Percentage	Centroid (PC1, PC2)
Potential Loyalists	0	42,865	45.9%	(−0.03, −0.89)
Champions	1	2,801	3.0%	(4.64, 0.40)
Hibernating	2	47,691	51.1%	(−0.24, 0.77)

6.4 What Changed — Before vs After Summary

Property	Before Clustering	After Clustering
Customer labels	None — 93,357 anonymous points in PCA space	Every customer assigned to one of 3 named segments
Visible structure	Continuous density gradient with one outlier cloud	Three distinct regions with centroids and boundaries
Actionability	No way to target customer groups differently	Each segment has a business profile for targeted campaigns
Centroid knowledge	No reference points	3 centroids define the 'average' customer in each segment
Noise handling	N/A	Zero noise points — every customer is segmented

6.5 Segment Profiles (Raw RFM Means)

While the clustering was performed in PCA space, the segment profiles below are computed on the original raw RFM values — giving business-interpretable metrics for each segment:

Metric	Potential Loyalists	Champions	Hibernating
Recency (days)	155	269	405
Frequency (orders)	1.0	2.0	1.0
Monetary (BRL)	154	295	156
Count	42,865	2,801	47,691

- **Potential Loyalists:** Recent one-time buyers (155 days). They bought once but recently — a re-engagement campaign could convert them to repeat buyers.
- **Champions:** The only repeat buyers (frequency = 2.0), with the highest monetary value (BRL 295). They are 3% of customers but disproportionately valuable. Retention and loyalty programmes should target this group.
- **Hibernating:** One-time buyers who last purchased 405 days ago. The largest segment (51.1%), representing lapsed customers who may need win-back offers or may have permanently churned.

Note: The similarity between Potential Loyalists and Hibernating on frequency (both 1.0) and monetary (BRL 154 vs 156) confirms that recency is the primary differentiator between these two groups — exactly what the PC2-axis split in the scatter plot shows.